

MDS5111: Python Programming

Course Project: Spatio-Temporal Logistics Optimization System (STLOS)

Corresponding Teaching Assistant: Ruixiang Wu (ruixiangwu@link.cuhk.edu.cn)

Term 2, 2026

Project Overview

Weight: 15% of Total Grade

Deadline: March 25th 23:59

Submission: Single `your_student_id.py` file via Blackboard.

Parameters: Fuel Cost: 7.5 RMB/L; Carbon Tax Multiplier: 0.1 per unit.

1 Background

You are appointed as a Data Engineer for a smart logistics firm in Shenzhen. The firm manages a heterogeneous fleet of vehicles (Electric, Diesel, Hybrid). Your task is to build a robust engine that ingests fragmented sensor telemetry and provides high-performance analytics on fuel efficiency and carbon footprint.

2 The Dataset

The firm stores data in a fragmented directory structure. You must navigate this using `pathlib`:

- `/district/vehicle_id/specs.json`: Metadata (`model`, `tank_capacity`, `carbon_rate`).
- `/district/vehicle_id/telemetry.csv`: Sensor data (`lat`, `lon`, `fuel_level`, `timestamp`).

3 Implementation Requirements

You must implement the `STLOS_Engine` class. Below are the technical specifications for the required methods:

Data Loading & File System

1. `_parse_specs(file_path)`: Parse a vehicle's JSON file. Return a dictionary.
2. `_load_telemetry(file_path)`: Load CSV telemetry. **Requirement:** The `timestamp` column must be cast to `datetime64[ns]`.
3. `ingest_fleet_data()`: Recursively crawl the dataset root. For each vehicle folder, initialize a `Vehicle` object and populate its `data` attribute with cleaned telemetry.

Data Wrangling & Cleaning

4. `clean_telemetry(df)`: Fill missing `lat/lon` values using linear interpolation. Euclidean interpolation is acceptable and expected here. Drop rows that remain `NaN`.
5. `remove_outliers(df, capacity)`: Remove records where `fuel_level > capacity` or `< 0`.
6. `align_time_series()`: Resample all vehicle data to a 5-minute (`'5min'`) frequency.
 - Use `.mean(numeric_only=True)` to aggregate and `.ffill()` to handle missing slots.
 - **Crucial**: Ensure the `district` string is preserved for each vehicle after resampling.
7. `merge_districts()`: Concatenate all individual vehicle DataFrames into a single `self.master_telemetry` DataFrame. Map the `v_id` and `c_rate` to this master table.

Numerical Analytics (NumPy Vectorization)

8. `compute_haversine_distance(lat1, lon1, lat2, lon2)`: Use the formula:

$$a = \sin^2\left(\frac{\Delta\phi}{2}\right) + \cos\phi_1 \cos\phi_2 \sin^2\left(\frac{\Delta\lambda}{2}\right), \quad d = 2R \cdot \text{atan2}(\sqrt{a}, \sqrt{1-a})$$

where $R = 6371$ km. This method **must** accept NumPy arrays and contain **no loops**.

9. `calculate_velocity_vector(v_id)`: Compute velocity $v = \frac{\Delta d}{\Delta t}$. *Note: Δt for a 5-min interval is 5/60 hours. Return a NumPy array of speeds in km/h.*
10. `detect_idling_events()`: Identify idling rows where $\Delta\text{lat} < 10^{-5}$, $\Delta\text{lon} < 10^{-5}$, and $\Delta\text{fuel_level} < 0$.
 - *Hint*: Real sensors have noise; do not use exact 0.0 comparisons for coordinates.
11. `compute_fuel_burn_rate()`: Use `groupby` and `shift` to calculate the distance delta (`dist_delta`) for every interval across the whole fleet.

Optimization & Reporting

12. `find_nearest_refuel_station(station_csv)`: Map every telemetry row to the nearest station index using **NumPy Broadcasting** to create a distance matrix of size $(N_{\text{telemetry}} \times M_{\text{stations}})$.
13. `calculate_fleet_carbon_tax()`: Formula: $T = (\sum(\text{dist_delta} \times \text{carbon_rate})) \times \text{multiplier}$.
 - *Hint*: This method depends on `dist_delta`. Write a safeguard to ensure `compute_fuel_burn_rate()` is called if the column is missing.
14. `generate_district_report()`: Return a Pivot Table (or GroupBy) with `district` as index and the **mean** of `fuel_level` as the performance value.
15. `export_final_stats(path)`: Save a JSON file with the following **exact** keys:
 - `"total_tax", "fleet_size", "avg_fuel_level", "total_idling_events"`.

4 Grading Rubrics (Total: 100 Points)

The Online Judge will evaluate your script based on three primary pillars:

Category	Weight	Criteria
Static Analysis	20 pts	Detection of <code>for/while</code> loops in methods 8–13. Full deduction (0/20) if any loop is found.
Functional Integrity	40 pts	Correctness of data loading, outlier removal, and OOP encapsulation.
Numerical Accuracy	40 pts	Final tax and fuel stats must match the hidden ground-truth within 10^{-2} tolerance.

- **File Naming:** You must rename your submission file to exactly your `9-digit-student-id.py` (e.g., `123456789.py`).
- **Interface Integrity: Strict Zero-Tolerance Policy:** Do not rename, modify, or delete any method, variable, or function names provided in the starter template. The autograder relies on these exact identifiers. Any script that fails to load due to missing or renamed methods will result in a **score of 0**.
- **Late Submission:** -10 pts per day.
- **Zero Tolerance:** Scripts that fail to import or contain syntax errors will receive a score of 0.